15 AI Engineering Projects

That Actually Land Jobs

A Step-by-Step Build Guide for Mid-Level Software Engineers
Transitioning into AI Engineering

BASWE | AiCareerOS™

How to Use This Guide

This guide contains fifteen complete project build plans. Each one is designed to take a mid-level software engineer with 3–5 years of production experience and produce a portfolio project that demonstrates AI engineering competence at a level hiring managers are actively looking for.

These are not tutorials. They are architectural blueprints with enough detail for you to build each system independently, making your own implementation decisions along the way. That's intentional — the decisions you make during the build are what you'll talk about in interviews.

What Makes These Projects Different

- Every project solves a real operational problem that AI teams face in production, not an academic exercise.
- They are designed to leverage your existing production engineering skills (Docker, CI/CD, APIs, databases) as a structural advantage.
- Each one demonstrates a different dimension of AI engineering competence: evaluation, cost optimization, observability, CI/CD integration, retrieval, fine-tuning, and multi-agent orchestration.
- None of them are "I called an LLM API and deployed a chatbot."

Estimated Timeline

Each project is scoped for roughly 12–14 days of focused work (2–3 hours per day). You don't need to build all fifteen. Pick the two or three that best align with the roles you're targeting and build those exceptionally well. Two deep, polished projects beat five shallow ones every time.

Project 1: Model Regression Detection System

What You're Building: A CI/CD-style pipeline that continuously tests any LLM-powered feature against a golden dataset whenever a prompt or model changes, detects quality regressions, and alerts your team via Slack before bad outputs reach users.

Why This Project Lands Interviews

Every AI team ships prompt changes blind. This project proves you think about what happens after deployment — the exact mindset hiring managers are desperate for and almost no candidates demonstrate.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Industry standard for ML tooling
LLM Provider	OpenAI API (gpt-4o / gpt-4o-mini)	Widely recognized; easy to swap later
Eval Framework	Custom + RAGAS or DeepEval	Shows you understand eval beyond accuracy
Data Storage	SQLite + JSON files	Zero infrastructure, portable, git-friendly
Alerting	Slack Webhooks	What real teams actually use
Scheduling	GitHub Actions	Runs on every PR; free tier is enough
Visualization	Streamlit or simple HTML report	Quick dashboard for diff views
Containerization	Docker	Shows production readiness

Step-by-Step Build Guide

Phase 1: Define the LLM Feature Under Test (Day 1–2)

1. **Build a simple LLM feature:** A customer support email classifier that reads an email and returns a category (billing, technical, account, general) plus a one-sentence summary. Wrap it in a single Python function with the prompt as a configurable parameter.
2. **Version your prompts:** Store prompts as versioned YAML files in a /prompts directory. Each file has a version ID, timestamp, the system prompt, and any few-shot examples. This is the “code” you’re running CI against.
3. **Create the interface contract:** Define a simple PromptConfig dataclass that your eval pipeline consumes. Input: email text. Output: structured JSON with category and summary. Keep it typed with Pydantic.

Phase 2: Build the Golden Dataset (Day 2–4)

1. **Curate 50–100 test cases by hand:** Write real-looking customer emails across all categories. For each one, write the correct category and an ideal summary. Do NOT

generate these with an LLM — the whole point is that these are human-verified ground truth.

2. **Include edge cases deliberately:** Add ambiguous emails that could be two categories, extremely short emails, emails with typos, emails in mixed languages, sarcastic emails. Label these with an "expected_difficulty" field.
3. **Store as versioned JSON:** Each test case gets a stable ID, the input, expected output, difficulty tag, and a notes field explaining why this case matters. Version the dataset file itself so you can track when the eval bar changes.

> **Interview Talking Point**
> When asked about this project, lead with how you built the golden dataset. Explain that you seeded it with hand-labeled data, then expanded it over time using failure cases. This signals you understand that evaluation quality is bounded by data quality — a production insight most candidates miss entirely.

Phase 3: Build the Evaluation Engine (Day 4–7)

1. **Create the test runner:** Write a function that takes a PromptConfig and the golden dataset, runs every test case through the LLM feature, and collects raw outputs. Use async batching to keep costs low and speed high.
2. **Implement multi-dimensional scoring:** Don't just check if the category matches. Score on: exact category match (binary), summary relevance (use an LLM-as-judge to rate 1–5), latency per request, and token usage. Store all dimensions per test case.
3. **Build the comparison logic:** The core value of this system is diffing. For every eval run, compare against the previous run. Calculate: overall pass rate delta, per-category accuracy delta, list of specific cases that flipped from pass to fail (regressions), and cases that flipped from fail to pass (improvements).
4. **Add statistical significance:** If 2 out of 80 cases flipped, is that signal or noise? Implement a simple threshold system: flag as warning if delta exceeds 3%, flag as critical if delta exceeds 8%. Make these configurable.

Phase 4: Build the Alerting and Reporting Layer (Day 7–9)

1. **Create the diff report:** Generate an HTML report that shows: run metadata (prompt version, model, timestamp), a summary scorecard comparing this run to the baseline, a table of every regressed case showing the old output vs. the new output side by side, and a trend chart showing scores over the last N runs.
2. **Wire up Slack alerts:** Use Slack's incoming webhook API. Send a structured message with: pass/warn/fail status, the headline numbers (e.g., "3 regressions detected, accuracy dropped from 94% to 89%"), and a link to the full HTML diff report.
3. **Add drift detection:** Beyond per-run diffs, track a rolling average of scores over time. If the 7-run moving average drops below a threshold even though no single run triggered

an alert, fire a “slow drift” warning. This catches gradual degradation that per-run checks miss.

Phase 5: Wire into CI/CD (Day 9–11)

1. **Create a GitHub Action workflow:** Trigger the eval pipeline on every PR that modifies files in the /prompts directory. The action should: run the eval, generate the report, post a summary comment on the PR with pass/fail status, and block merge if critical regressions are detected.
2. **Containerize everything:** Write a Dockerfile that packages the eval runner, the golden dataset, and the reporting layer. The container should accept environment variables for the LLM API key, Slack webhook URL, and threshold configs.
3. **Write a README that reads like internal documentation:** Include: a one-paragraph summary of what this does, setup instructions, how to add new test cases to the golden dataset, how to adjust thresholds, and architecture decisions with rationale. Do NOT write it like a tutorial. Write it like onboarding docs for a new teammate joining your team.

Phase 6: Polish for Portfolio (Day 11–12)

1. **Record a 3-minute Loom walkthrough:** Show the system running end to end: change a prompt, trigger the eval, show the Slack alert, walk through the diff report. This is more persuasive than any README.
2. **Write a short blog post or README section:** Explain the problem (teams ship prompt changes blind), your approach (CI/CD for model behavior), and one specific design decision you’re proud of (e.g., why you track slow drift separately from per-run regressions).

Project 2: LLM Cost Autopilot

What You’re Building: An intelligent routing layer that sits in front of multiple LLM providers, analyzes each incoming request’s complexity, routes it to the cheapest model capable of handling it at acceptable quality, and continuously validates that routing decisions are correct.

> **Why This Project Lands Interviews**
> Every company running LLMs at scale is bleeding money on over-provisioned model calls. Building a cost optimizer signals that you understand AI engineering as a business problem, not just a technical one — and that’s the gap between a junior hire and a senior one.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Ecosystem compatibility

LLM Providers	OpenAI, Anthropic, Ollama (local)	Mix of cloud and local models
Router	FastAPI	Async-native, production-grade
Classifier	Scikit-learn or small fine-tuned model	Lightweight complexity scoring
Eval	Custom scoring + LLM-as-judge	Quality verification loop
Logging	SQLite + structured JSON logs	Full audit trail per request
Dashboard	Streamlit or Grafana	Cost and quality visualization
Containerization	Docker + docker-compose	Multi-service orchestration

Step-by-Step Build Guide

Phase 1: Build the Unified Model Interface (Day 1–3)

1. **Create a model registry:** Define a ModelConfig dataclass with: provider name, model ID, cost per input token, cost per output token, average latency, and a quality tier (high/medium/low). Populate it with real pricing for GPT-4o, GPT-4o-mini, Claude Sonnet, Claude Haiku, and a local Llama model via Ollama.
2. **Build the abstraction layer:** Write a single send_request(prompt, model_config) function that handles the provider-specific API calls behind a unified interface. Every call returns a standardized Response object with: the output text, tokens used (input + output), latency, cost, and the model ID.
3. **Test every provider:** Send the same 10 prompts to every model in your registry. Log the outputs, costs, and latencies. This gives you baseline data for the routing logic and validates your abstraction layer works.

Phase 2: Build the Complexity Classifier (Day 3–6)

1. **Define complexity tiers:** Create three tiers. Tier 1 (simple): reformatting, extraction, basic Q&A from provided context. Tier 2 (moderate): summarization, classification, structured analysis. Tier 3 (complex): multi-step reasoning, creative generation, nuanced judgment calls.
2. **Build a labeled dataset:** Write 200+ example prompts across all three tiers. Label each one by hand. Include features you'll extract: token count, presence of instructions like "analyze" or "compare," number of constraints, whether context is provided, and output format complexity.
3. **Train the classifier:** Start with a simple scikit-learn model (logistic regression or random forest) on your extracted features. You're not optimizing for classifier perfection — you're building the routing skeleton. Track accuracy and confusion matrix. Anything above 80% accuracy on a held-out set is fine for V1.

4. **Create the routing map:** Map each complexity tier to a model. Tier 1 → cheapest model (Haiku or local Llama). Tier 2 → mid-tier (GPT-4o-mini or Sonnet). Tier 3 → highest quality (GPT-4o or Opus). Store this as a configurable YAML so you can swap models without code changes.

Phase 3: Build the Async Quality Verification Loop (Day 6–9)

1. **Define quality thresholds per use case:** For each request type, define what "good enough" means. For extraction tasks: did it get all the key fields? For summarization: LLM-as-judge score above 4/5. For classification: does the label match what GPT-4o would have said?
2. **Build the async verifier:** After the response is returned to the user, queue an async job that sends the same prompt to the highest-tier model and compares outputs. Score the agreement. If the cheap model's output diverges significantly, log it as a routing failure.
3. **Implement auto-escalation:** If the verifier catches a failure, automatically re-run the request with the higher-tier model and return the better result (if latency permits). Log the escalation event with: original model, escalated model, cost delta, and the quality gap that triggered it.
4. **Feed failures back to the classifier:** Every routing failure becomes a new training example for the complexity classifier. Build a simple feedback loop that retrains the classifier weekly using accumulated failure data. This is the flywheel that makes the system get smarter over time.

Phase 4: Build the Logging and Cost Dashboard (Day 9–11)

1. **Log everything:** Every request gets a row in your database with: timestamp, prompt hash, complexity tier, routed model, cost, latency, quality score from verifier, and whether it was escalated. This is your audit trail.
2. **Build the cost dashboard:** Show: total cost per day/week with a comparison to what it would have cost using GPT-4o for everything ("you saved $X"), routing distribution (pie chart of which models handle what percentage), quality score distribution, and escalation rate over time.
3. **Add the money shot metric:** Calculate and prominently display the cost reduction percentage. If routing to cheaper models saved 60% compared to sending everything to the most expensive model, that number is the headline of your portfolio piece.

Phase 5: Expose as an API (Day 11–13)

1. **Build the FastAPI service:** A single POST /v1/completions endpoint that accepts a standard chat completion request. The user doesn't choose the model — the router does. Return the response with metadata showing which model was selected and why.

2. **Add configuration endpoints:** GET /v1/models (list available models and their costs), GET /v1/stats (cost savings summary), and PUT /v1/routing-config (update tier-to-model mappings without redeploying).
3. **Containerize and document:** Docker-compose with the API service, a background worker for async verification, and the SQLite database. Write a README with architecture diagram, setup instructions, and the cost savings number front and center.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Run a realistic load test:** Send 500–1,000 diverse prompts through the system. Generate the final cost savings report. Screenshot the dashboard. These artifacts are what you put in your portfolio.
2. **Write the case study:** Frame it as: "I built a system that reduced LLM API costs by X% while maintaining Y% quality parity." Lead with the number. Explain the routing logic. Show the feedback loop. This is a story a VP of Engineering immediately understands.

Project 3: Failure Forensics Tool for AI Pipelines

What You're Building: An observability layer for multi-step AI pipelines that traces every intermediate step, identifies exactly where failures originate when the final output is bad, and feeds flagged failures back into a growing evaluation dataset.

> **Why This Project Lands Interviews**
> When a multi-step AI pipeline produces garbage, most teams have no idea which step broke. You're building the tool that answers "where did this go wrong?" — essentially a mini LangSmith/Braintrust. Being able to articulate why observability matters for AI systems is a senior-level signal.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Standard for ML tooling
Pipeline Framework	Custom chain or LangChain	Demonstrates orchestration knowledge
LLM Provider	OpenAI API	Widely available
Tracing	OpenTelemetry + custom spans	Industry-standard observability
Storage	SQLite + JSON trace files	Simple, inspectable, git-friendly
Visualization	React frontend or Streamlit	Interactive trace explorer
Feedback Loop	Simple REST API	Humans flag bad outputs

Containerization	Docker	Production packaging

Step-by-Step Build Guide

Phase 1: Build the Multi-Step Pipeline (Day 1–3)

1. **Design a 4-step pipeline:** Step 1 (Intake): Accept a raw document (text, markdown, or simulated PDF text). Step 2 (Extraction): Use an LLM to extract structured entities (names, dates, amounts, key terms). Step 3 (Classification): Classify the document type (contract, invoice, report, correspondence). Step 4 (Summarization): Generate a structured summary tailored to the document type.
2. **Make each step a clean, isolated function:** Each step takes a typed input and returns a typed output. Use Pydantic models for every intermediate data structure. This is critical — if your steps are spaghetti, your tracing will be meaningless.
3. **Inject realistic failure modes:** Deliberately include documents that will break things: a contract with no dates, an invoice with amounts in different currencies, a document that's ambiguously between two categories. You need failures to trace.

Phase 2: Build the Tracing Layer (Day 3–6)

1. **Create a Trace object:** Every pipeline execution gets a unique trace_id. The Trace contains: a list of Span objects (one per step), the final output, and a status (success/failure/degraded).
2. **Instrument each step with spans:** Wrap each pipeline step in a context manager that automatically captures: step name, input (serialized), output (serialized), LLM prompt sent, LLM raw response, token count, latency, and any errors. Use a decorator pattern so instrumenting a new step is one line of code.
3. **Add confidence scoring at each step:** After each LLM call, have the model also output a confidence score (1–5) for its own response. Store this in the span. When you're tracing backward from a failure, low-confidence spans are your primary suspects.
4. **Store traces as structured JSON:** Write each complete trace to a JSON file and index it in SQLite (trace_id, timestamp, status, final_score). This gives you both human-readable traces and queryable metadata.

Phase 3: Build the Backward Trace Analyzer (Day 6–9)

1. **Implement root cause analysis logic:** When a trace is flagged as failed, walk backward through the spans. At each step, ask: is this step's output a reasonable transformation of its input? Use an LLM-as-judge to score each step's output quality given its input. The first step with a significant quality drop is your root cause.
2. **Categorize failure types:** Create a taxonomy: Extraction Hallucination (extracted entities that don't exist in the source), Misclassification (wrong document type), Propagation Error (step N was correct but step N+1 misinterpreted its output), Prompt Failure (the

LLM ignored instructions), and Context Loss (important information from earlier steps was dropped).

3. **Build the evidence chain:** For each diagnosed failure, produce a structured explanation: “Step 2 (Extraction) hallucinated the entity ‘John Smith’ which does not appear in the source document. This propagated to Step 4, which included it in the summary.” Include the specific input/output pairs as evidence.

Phase 4: Build the Visual Trace Explorer (Day 9–11)

1. **Create the trace view:** A visual representation of the pipeline where each step is a node. Color-code by status: green (healthy), yellow (low confidence), red (identified root cause). Clicking a node shows the full span details — input, output, prompt, confidence score.
2. **Add the diff view:** For failed traces, show a side-by-side comparison: what the step received vs. what it produced vs. what it should have produced (based on the golden dataset or human correction). Highlight the specific divergence.
3. **Build the flagging interface:** A simple button that lets a user mark any trace as “bad output.” When clicked, the system runs the backward trace analysis and displays the root cause diagnosis. The user can confirm or override the diagnosis.

Phase 5: Build the Feedback-to-Eval Loop (Day 11–13)

1. **Auto-generate eval cases from flags:** Every time a human flags a bad output and confirms the root cause, automatically create a new test case: the original input, the failing step, the bad output, the human-corrected output, and the failure category. Append it to a growing eval dataset.
2. **Build regression tracking:** Periodically re-run the accumulated eval dataset against the current pipeline. Track whether known failure cases are still failing or have been fixed. Show a trend line of “known issues resolved” over time.
3. **Create failure analytics:** Dashboard showing: most common failure types, which pipeline step fails most often, failure rate over time, and average time to root cause. This is the data that tells a product team where to invest engineering effort.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Prepare a demo scenario:** Process 50 documents through the pipeline. Ensure at least 8–10 produce failures of different types. Record a Loom showing: a bad output, opening the trace explorer, diagnosing the root cause, flagging it, and showing the eval dataset growing.
2. **Write the architecture doc:** Include a diagram of the pipeline with the tracing layer, the backward analysis flow, and the feedback loop. Frame it as: “I built an observability system that reduces mean time to root cause for AI pipeline failures from hours of manual debugging to seconds of automated diagnosis.”

Project 4: Self-Healing Technical Documentation

What You're Building: A GitHub Action that monitors a codebase, detects when code changes make documentation inaccurate, identifies the specific stale sections, and either auto-generates a PR with corrected docs or flags the discrepancies for human review.

> **Why This Project Lands Interviews**
> This project lives inside a CI/CD pipeline, not a Streamlit demo. It solves a universal engineering pain point that every interviewer has personally experienced. And it demonstrates the full AI engineering stack — embeddings, retrieval, LLM generation, and production deployment — in a system that other engineers would actually want to install.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+ / TypeScript	Your choice; both work in GitHub Actions
Embeddings	OpenAI text-embedding-3-small	Cheap, fast, high quality
Vector Store	ChromaDB (file-based)	No server needed; persists to disk
LLM	GPT-4o or Claude Sonnet	Strong code understanding
Git Integration	PyGithub + git diff	PR creation and diff parsing
CI/CD	GitHub Actions	Native integration, free tier
Containerization	Docker (for the Action)	Reproducible runs

Step-by-Step Build Guide

Phase 1: Build the Code-to-Docs Mapping (Day 1–4)

1. **Parse the codebase into semantic chunks:** Write a parser that walks through the codebase and extracts meaningful units: function signatures with docstrings, class definitions, API endpoint definitions, configuration schemas, and CLI command definitions. Each chunk gets a stable identifier (file path + function/class name).
2. **Parse documentation into sections:** Split markdown docs into sections by heading. Each section gets: its heading path (e.g., "Configuration > Environment Variables"), the raw content, and a list of code references it mentions (function names, class names, config keys, CLI commands).
3. **Build the link graph:** Create explicit links between doc sections and code chunks. Start with simple heuristics: if a doc section mentions a function name that exists in the codebase, link them. Then enhance with embeddings: compute embeddings for both

code chunks and doc sections, and link any pairs with cosine similarity above a threshold.
4. **Store the mapping:** Persist the code-to-docs graph as a JSON file in the repo. This is your index. When code changes, you'll query this graph to find which doc sections might be affected.

Phase 2: Build the Change Detection Pipeline (Day 4–7)

1. **Parse the git diff:** On every PR, extract the list of changed files and the specific changes (added/removed/modified lines). Map each change to the code chunks it affects using your code parser.
2. **Filter for meaningful changes:** Not every code change affects docs. Skip: comment-only changes, whitespace changes, internal refactors that don't change behavior, and test file changes. Focus on: API signature changes, configuration changes, new/removed features, and behavioral changes to existing functions.
3. **Identify affected doc sections:** For each meaningful code change, query your code-to-docs graph to find linked documentation sections. These are your "suspects" — sections that might now be stale.
4. **Verify staleness with an LLM:** For each suspect doc section, send the LLM: the old code, the new code, and the doc section content. Ask it to determine whether the documentation is still accurate given the code change. If not, ask it to explain specifically what's wrong. This step filters out false positives.

Phase 3: Build the Doc Repair Engine (Day 7–10)

1. **Generate targeted corrections:** For each confirmed stale section, send the LLM: the current doc section, the new code, and the staleness diagnosis from Phase 2. Ask it to rewrite only the stale parts, preserving the original style, tone, and structure. Explicitly instruct it not to rewrite parts that are still accurate.
2. **Validate the corrections:** Run a second LLM pass that checks: does the corrected doc accurately describe the new code? Did it preserve the parts that were already correct? Is the writing style consistent with the rest of the document? This is your quality gate before creating a PR.
3. **Handle different correction modes:** Not all staleness is the same. For simple changes (renamed parameter, updated default value), auto-fix with high confidence. For complex changes (new feature, removed capability), generate a draft with clear TODO markers and request human review. Let the confidence level determine the mode.

Phase 4: Build the GitHub Action (Day 10–12)

1. **Package as a GitHub Action:** Create a Dockerfile and action.yml that defines the Action's inputs (LLM API key, confidence threshold, auto-merge for high-confidence

fixes), outputs (list of stale sections found, corrections generated), and triggers (runs on PRs that modify code files).
2. **Implement the PR workflow:** For high-confidence fixes: create a new branch, apply the corrections, open a PR with a clear description of what changed and why. For low-confidence flags: add a comment on the original PR listing the doc sections that need human review, with links to the specific sections.
3. **Add a PR comment summary:** On every PR that triggers the Action, post a comment: "Doc Check Results: 3 sections verified accurate, 1 auto-fixed (see PR #42), 2 flagged for review." Include links to everything. This is what makes the tool feel integrated and professional.

Phase 5: Test on a Real Repository (Day 12–13)

1. **Fork a real open-source project:** Pick a well-documented project (FastAPI, Pydantic, or similar). Fork it, install your Action, and deliberately make code changes that should invalidate docs. Test that the Action correctly identifies the staleness and generates reasonable fixes.
2. **Measure accuracy:** Across your test cases, track: true positives (correctly identified stale docs), false positives (flagged accurate docs as stale), false negatives (missed actual staleness), and correction quality (are the fixes actually right?). Report these numbers in your README.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Make it installable:** Publish to the GitHub Actions marketplace (it's free). Having an Action that other people can actually install is a fundamentally different portfolio signal than a repo that just sits there.
2. **Record the demo:** Show: a code change being pushed, the Action running, the PR comment appearing, and the auto-generated doc fix PR. Keep it under 3 minutes. Lead with the problem ("every team's docs are perpetually stale") and end with the result.

Project 5: LLM Output Arbitration System

What You're Building: A multi-agent pipeline that takes any LLM-generated output, routes it to multiple competing critic models that independently evaluate it for accuracy, consistency, and completeness, then synthesizes their critiques into a single confidence-scored verdict with actionable callouts.

Why This Project Lands Interviews

Instead of building yet another system that generates answers, you're building one that catches bad answers. This flips the script on every other portfolio project out there and demonstrates the evaluation mindset that AI teams are actively hiring for but rarely see in candidates.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Ecosystem standard
Agent Framework	LangGraph	Top-requested framework in AI eng roles
LLM Providers	OpenAI + Anthropic + Ollama	Multi-provider comparison is the point
Structured Output	Pydantic + instructor library	Type-safe LLM outputs
Storage	SQLite + JSON	Audit trail for every arbitration
API	FastAPI	Production-grade serving
Visualization	React or Streamlit	Verdict explorer UI

Step-by-Step Build Guide

Phase 1: Design the Critic Agent Architecture (Day 1–3)

1. **Define the evaluation dimensions:** Create three specialized critic roles. The Factual Accuracy Critic checks whether claims are verifiable and internally consistent. The Logical Consistency Critic checks whether the reasoning follows and conclusions are supported. The Completeness Critic checks whether the response addresses all parts of the question and flags gaps.
2. **Design the structured critique format:** Each critic returns a Pydantic model with: dimension (accuracy/logic/completeness), score (1–5), list of specific issues found (each with a quote from the original, the problem description, and severity), and an overall confidence in their own assessment. Use the instructor library to enforce structured outputs from every model.
3. **Assign different models to different critics:** This is deliberate. Route the accuracy critic through GPT-4o, the logic critic through Claude, and the completeness critic through a local Llama model. The disagreements between models are the most valuable signal. If all three used the same model, they'd share the same blind spots.

Phase 2: Build the Orchestration Layer with LangGraph (Day 3–6)

1. **Define the graph:** Create a LangGraph state graph with nodes for: input parsing, parallel critic dispatch, critique collection, disagreement detection, adjudication, and verdict

synthesis. The graph should handle the full lifecycle from receiving an output to delivering a verdict.

2. **Implement parallel critic dispatch:** All three critics should run in parallel, not sequentially. Use LangGraph's branching to fan out to all critics simultaneously and fan in when all results are collected. This keeps latency reasonable and mirrors how real multi-agent systems are built.
3. **Build the disagreement detector:** After collecting all critiques, compare them. Flag cases where critics disagree on whether something is an issue, where severity ratings differ by more than 2 points, or where one critic found issues the others missed entirely. These disagreements are the interesting cases.
4. **Handle edge cases in the graph:** What if one critic's API call fails? Implement retry logic and graceful degradation — the system should still produce a verdict from the remaining critics, with a note that one dimension has lower confidence. What if all critics agree it's perfect? Short-circuit the adjudicator and return a high-confidence pass.

Phase 3: Build the Adjudicator Agent (Day 6–8)

1. **Design the adjudication prompt:** The adjudicator receives: the original output being evaluated, all three critic reports, and the list of detected disagreements. Its job is to weigh the evidence, resolve conflicts, and produce a final verdict. Prompt it to reason through each disagreement explicitly.
2. **Implement evidence-based resolution:** When critics disagree about a factual claim, the adjudicator should attempt to verify it. When they disagree on logic, the adjudicator should trace the reasoning chain step by step. When they disagree on completeness, the adjudicator should re-read the original question and determine what was actually required.
3. **Produce the final verdict:** A structured output with: overall quality score (1–10), confidence level, a list of confirmed issues (with severity and evidence), a list of dismissed flags (issues one critic raised but the adjudicator overruled, with reasoning), and a one-paragraph summary of the assessment.

Phase 4: Build the Verdict Explorer UI (Day 8–10)

1. **Create the main verdict view:** Display the original output with inline annotations. Each flagged issue should be highlighted in the text with a colored marker (red for confirmed issues, yellow for low-confidence flags, green for explicitly validated claims). Clicking a marker shows the full evidence chain.
2. **Build the critic comparison panel:** A side-by-side view showing all three critics' assessments next to each other. Highlight agreements in green and disagreements in orange. This visual makes the multi-agent architecture immediately comprehensible to anyone reviewing your portfolio.

3. **Add a batch mode:** Allow submitting multiple outputs for arbitration. Show results in a sortable table with: output excerpt, overall score, number of issues found, and confidence. This demonstrates the system works at scale, not just for one-off demos.

Phase 5: Expose as an API and Add Analytics (Day 10–12)

1. **Build the FastAPI service:** POST /v1/arbitrate accepts an LLM output (and optionally the original prompt) and returns the full verdict. POST /v1/arbitrate/batch accepts multiple outputs. GET /v1/arbitrations/{id} retrieves a past verdict. Keep the API clean and well-documented with OpenAPI specs.
2. **Add analytics on critic behavior:** Over many arbitrations, track: which critic finds the most issues, which critic is overruled by the adjudicator most often, which failure types are most common, and how often critics agree vs. disagree. This meta-analysis is portfolio gold.
3. **Containerize everything:** Docker-compose with the FastAPI service, the LangGraph pipeline, and the SQLite database. Include a docker-compose.yml that spins up Ollama locally for the completeness critic so reviewers can run the full system without paid API keys for all three providers.

Phase 6: Polish for Portfolio (Day 12–14)

1. **Prepare compelling test cases:** Run the system against: a factually incorrect LLM response (planted errors), a logically flawed argument, a response that technically answers the question but misses the point, and a genuinely good response (to show it can give a clean bill of health). Screenshot the verdicts for all four.
2. **Write the narrative:** Frame it as: “I built a system where AI models audit each other’s work. Three specialized critics independently evaluate any LLM output, and an adjudicator resolves their disagreements into a single verdict.” Lead with the architecture diagram showing the parallel fan-out pattern. End with the analytics showing that multi-model critique catches issues that single-model self-evaluation misses.

Project 6: RAG Pipeline with Hybrid Search Over Internal Docs

What You’re Building: A production-grade Retrieval-Augmented Generation system that ingests a company’s internal documentation, indexes it with both dense vector and sparse keyword search, retrieves the most relevant context for any question, and generates grounded answers with inline source citations.

Why This Project Lands Interviews

RAG is the single most requested skill in AI engineering job descriptions. But most candidates build a toy demo with a single PDF. You're building a system with hybrid retrieval, chunking strategy decisions, and citation verification — the production concerns that separate a real RAG engineer from someone who followed a LangChain quickstart.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Ecosystem standard
Embeddings	OpenAI text-embedding-3-small	Cost-effective, high quality
Vector Store	ChromaDB or Qdrant	File-based or containerized
Sparse Search	BM25 via rank_bm25	Keyword matching for exact terms
LLM	GPT-4o or Claude Sonnet	Strong grounding and citation
Chunking	LangChain text splitters	Configurable overlap and size
API	FastAPI	Async-native, production-grade
Containerization	Docker	Reproducible deployment

Step-by-Step Build Guide

Phase 1: Build the Ingestion and Chunking Pipeline (Day 1–3)

1. **Build a multi-format document loader:** Accept markdown, text, HTML, and PDF files. Normalize everything into clean plaintext with metadata (source file, section heading, page number). Store raw documents alongside processed versions so you can re-index without re-uploading.
2. **Implement configurable chunking:** Build three chunking strategies and make them switchable: fixed-size with overlap (baseline), recursive character splitting by section headers (structure-aware), and semantic chunking that splits on topic boundaries using embedding similarity. Track which strategy each chunk used.
3. **Generate and store embeddings:** Embed every chunk using text-embedding-3-small. Store in ChromaDB with metadata: source document, chunk index, section heading, chunking strategy, and character count. Build the BM25 index in parallel over the same chunks. Both indexes must stay in sync.
4. **Add deduplication:** Before inserting a chunk, check for near-duplicates (cosine similarity > 0.95 against existing chunks). Flag and skip duplicates. This prevents the retriever from wasting context window slots on redundant content when the same information appears in multiple docs.

Phase 2: Build the Hybrid Retrieval Engine (Day 3–6)

1. **Implement dense retrieval:** Query the vector store with the embedded user question. Return the top-k chunks ranked by cosine similarity. Start with k=10.

2. **Implement sparse retrieval:** Run the same query through BM25 over the chunk corpus. Return top-k by BM25 score. This catches exact keyword matches that semantic search might miss — critical for technical documentation with specific function names, config keys, or error codes.
3. **Build the fusion layer:** Implement Reciprocal Rank Fusion (RRF) to combine dense and sparse results into a single ranked list. RRF assigns scores based on rank position across both lists and merges them. Make the weighting configurable (e.g., 0.7 dense / 0.3 sparse) so you can tune it per use case.
4. **Add a reranker:** After fusion, send the top 20 candidates through a cross-encoder reranker (use a small model or LLM-as-judge) that scores each chunk's relevance to the actual question. Keep the top 5. This second pass dramatically improves precision and is a strong interview talking point.

Phase 3: Build the Generation and Citation Layer (Day 6–9)

1. **Design the grounded generation prompt:** Construct a system prompt that instructs the LLM to answer only from the provided context, cite specific chunks using bracketed references ([1], [2]), and explicitly state when the context doesn't contain enough information to answer. Include the retrieved chunks as numbered context blocks.
2. **Implement citation verification:** After generation, parse the model's citations and verify each one. Does [1] actually support the claim it's attached to? Send each citation-claim pair to an LLM-as-judge for verification. Flag unsupported citations. This is the quality layer most RAG systems skip entirely.
3. **Build the answer confidence scorer:** Score each answer on: retrieval confidence (how relevant were the top chunks?), citation coverage (what percentage of claims have verified citations?), and answer completeness (did the response address all parts of the question?). Return a composite confidence score alongside the answer.
4. **Handle the "I don't know" case gracefully:** If retrieval confidence is below a threshold, don't hallucinate. Return a structured response that says what the system found, what it couldn't find, and which documents might be worth checking manually. This is more useful than a fabricated answer and signals production maturity.

Phase 4: Build the Evaluation Framework (Day 9–11)

1. **Create a golden Q&A dataset:** Write 50+ question-answer pairs by hand, each tied to specific sections of your document corpus. Include straightforward lookups, multi-hop questions (answer requires combining information from two documents), questions with no answer in the corpus, and ambiguous questions.
2. **Implement automated eval metrics:** For each test case, measure: answer correctness (LLM-as-judge against golden answer), faithfulness (are all claims grounded in retrieved context?), retrieval relevance (were the right chunks retrieved?), and citation accuracy (do citations actually support claims?). Run the full suite on every pipeline change.
3. **Build a chunking strategy comparison:** Run the same eval suite across your three chunking strategies. Generate a comparison report showing which strategy wins on which metrics. This data drives your architecture decisions and gives you concrete numbers for interviews.

Phase 5: Expose as an API and Dashboard (Day 11–13)

1. **Build the FastAPI service:** POST /v1/ask accepts a question and returns the answer with citations, confidence scores, and source metadata. GET /v1/documents lists indexed documents. POST /v1/ingest accepts new documents for indexing. Include OpenAPI documentation.
2. **Build a simple query dashboard:** A Streamlit or React frontend where you can ask questions and see: the generated answer with clickable citations, the retrieved chunks ranked by relevance, confidence scores broken down by dimension, and a toggle to compare hybrid vs. dense-only retrieval side by side.
3. **Containerize everything:** Docker-compose with the API service, ChromaDB, and the frontend. Include a seed script that indexes a sample documentation corpus so reviewers can spin it up and test immediately.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record a demo walkthrough:** Show: ingesting a set of documents, asking questions of varying difficulty, the citation verification catching a hallucination, and the hybrid vs. dense-only comparison. Keep it under 4 minutes.
2. **Write the case study:** Frame it as: "I built a RAG system with hybrid search that achieves X% faithfulness and Y% citation accuracy on a 50-question eval suite." Lead with the numbers. Explain why hybrid beats dense-only for technical documentation. Show the chunking strategy comparison data.

Project 7: Semantic Caching Layer for LLM APIs

What You're Building: A middleware caching service that sits between your application and any LLM provider, detects semantically similar requests that have already been answered, and serves cached responses instantly — cutting latency to near-zero and reducing API costs by 30–60% on typical workloads.

> **Why This Project Lands Interviews**
> Every company running LLMs at scale has the same problem: redundant API calls burning money and adding latency. Building a semantic cache proves you think about infrastructure efficiency, not just model accuracy — and it's the kind of system that engineering managers immediately understand the ROI of.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Ecosystem compatibility
Embeddings	OpenAI text-embedding-3-small	Fast, cheap, high quality
Vector Store	Redis + RedisVL or Qdrant	Sub-millisecond lookups
Proxy Layer	FastAPI	Drop-in API replacement

Cache Policy	Custom TTL + similarity threshold	Tunable hit rate vs. freshness
Monitoring	Prometheus + Grafana	Real-time hit rate dashboards
Containerization	Docker + docker-compose	Multi-service orchestration

Step-by-Step Build Guide

Phase 1: Build the Cache Index and Similarity Engine (Day 1–3)

1. **Design the cache key strategy:** You can't use exact string matching — "What is Python?" and "Explain Python to me" should hit the same cache entry. Embed every incoming prompt using text-embedding-3-small and store the embedding alongside the response in your vector store.
2. **Build the similarity lookup:** On every incoming request, embed the prompt and query the vector store for the nearest neighbor. If cosine similarity exceeds a configurable threshold (start at 0.95), it's a cache hit. Return the stored response. If below threshold, it's a miss — forward to the LLM provider.
3. **Implement cache storage:** After a cache miss, forward the request to the actual LLM, capture the full response (including token counts, model ID, and finish reason), and store it alongside the prompt embedding. Include metadata: timestamp, TTL expiry, hit count, and the original prompt text for debugging.
4. **Handle system prompt and parameters:** Two identical user prompts with different system prompts or temperature settings should NOT share cache entries. Incorporate the system prompt hash and key generation parameters (temperature, max_tokens, model) into the cache key. This prevents cross-contamination between different use cases.

Phase 2: Build the Drop-In Proxy API (Day 3–5)

1. **Mirror the OpenAI API contract:** Build a FastAPI service that accepts the exact same request format as the OpenAI chat completions endpoint. This means any application can switch to your cache proxy by just changing the base URL — zero code changes. Return the same response format with an additional header indicating cache hit/miss.
2. **Add provider routing:** Behind the proxy, support multiple LLM providers (OpenAI, Anthropic, Ollama). Route based on the model field in the request. The cache layer is provider-agnostic — a cached OpenAI response can't be served for an Anthropic request, but the cache logic itself is shared.
3. **Implement streaming support:** Cache hits should return instantly (no streaming needed). Cache misses should stream the response from the provider to the client while simultaneously buffering it for cache storage. This is the tricky part — you need to handle partial responses and only cache complete, successful responses.

Phase 3: Build Cache Policies and Eviction (Day 5–8)

1. **Implement TTL-based expiration:** Every cache entry gets a configurable TTL. For factual/stable queries, set long TTLs (24h+). For queries that reference time or current events, set short TTLs (1h) or disable caching entirely. Build a classifier that auto-assigns TTL tiers based on prompt content.
2. **Add cache invalidation triggers:** When the system prompt for a feature changes, invalidate all cache entries associated with that system prompt hash. When a model is upgraded (e.g., GPT-4o to a newer version), invalidate entries for that model. Provide API endpoints for manual invalidation by prefix or tag.
3. **Build the similarity threshold tuner:** Expose an endpoint that lets you test different similarity thresholds against historical data. Show: at 0.90 threshold, hit rate is X% but Y% of hits returned slightly wrong answers. At 0.98 threshold, hit rate drops to Z% but accuracy is near-perfect. This tradeoff visualization is the core interview talking point.
4. **Implement adaptive thresholds:** Different request types have different tolerance for approximate matches. Classification tasks can tolerate lower similarity (0.90) because the answer space is constrained. Creative generation tasks need higher similarity (0.98) or should skip caching entirely. Let the system learn these thresholds from feedback.

Phase 4: Build Monitoring and Analytics (Day 8–10)

1. **Instrument everything with Prometheus metrics:** Track: cache hit rate (overall and per-model), average latency for hits vs. misses, estimated cost savings per hour/day, cache size and eviction rate, and similarity score distribution for hits and near-misses. Export as Prometheus metrics.
2. **Build the Grafana dashboard:** Create panels showing: real-time hit rate, cumulative cost savings (the money shot), latency comparison (cached vs. uncached P50/P95/P99), cache capacity utilization, and a time series of the similarity threshold's effect on hit rate over the past 7 days.
3. **Add a near-miss analyzer:** Track queries that fell just below the similarity threshold. These are potential cache hits if the threshold were slightly lower. Analyzing near-misses lets you tune the threshold with real data and identify opportunities for query normalization (e.g., stripping filler words before embedding).

Phase 5: Containerize and Load Test (Day 10–12)

1. **Docker-compose the full stack:** Services: FastAPI proxy, Redis (with RedisVL), Prometheus, Grafana, and an optional Ollama instance for local model testing. Include pre-configured Grafana dashboards so reviewers see data immediately.
2. **Run a realistic load test:** Send 2,000+ requests through the system using a mix of unique and repeated queries (simulating realistic usage patterns). Measure: hit rate convergence over time, latency percentiles, and total cost savings. These numbers are your portfolio headline.

3. **Write the README as an internal proposal:** Frame it as: “Here’s what this would save us if deployed in front of our LLM calls.” Include the load test results, the cost savings projection, and a deployment guide. This isn’t a tutorial — it’s a business case with code attached.

Phase 6: Polish for Portfolio (Day 12–14)

1. **Record the demo:** Show: a fresh query hitting the LLM (with latency), the same query hitting the cache (instant), a semantically similar query also hitting the cache, and the Grafana dashboard showing cost savings accumulate in real time.
2. **Lead with the number:** “A drop-in caching layer that reduced LLM API costs by X% and P95 latency by Y% on a 2,000-request load test.” That’s the headline. Everything else is supporting evidence.

Project 8: Text-to-SQL Interface with Guardrails and Hallucination Detection

What You’re Building: A natural language interface that translates plain English questions into SQL queries against a real database, executes them safely with guardrails preventing destructive operations, validates that the generated SQL actually answers the question asked, and presents results with a confidence score.

> **Why This Project Lands Interviews**
> Text-to-SQL is one of the highest-value applications of LLMs in enterprise settings, and it’s notoriously hard to get right. Building one with guardrails and hallucination detection proves you can ship AI features that a compliance team would actually approve — which is the bar for production AI at any serious company.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Standard for data tooling
LLM	GPT-4o or Claude Sonnet	Strong structured output
Database	PostgreSQL or DuckDB	Real SQL engine, not SQLite toy
Schema Extraction	SQLAlchemy	Automatic schema introspection
Guardrails	Custom middleware	Prevents destructive queries
Validation	LLM-as-judge + result check	Hallucination detection
API	FastAPI	Production-grade serving
Containerization	Docker + docker-compose	DB + API orchestration

Step-by-Step Build Guide

Phase 1: Build the Schema-Aware Prompt Engine (Day 1–3)

1. **Auto-extract the database schema:** Use SQLAlchemy to introspect the database and produce a structured representation: tables, columns with types, primary/foreign key relationships, and sample values for categorical columns. This becomes the context the LLM uses to write SQL.
2. **Build the dynamic prompt constructor:** For each user question, assemble a prompt with: the relevant schema (not the entire database — filter to tables likely needed), foreign key relationships, sample values for disambiguation, and any column descriptions or business glossary terms. Include 3–5 few-shot examples of question-to-SQL pairs specific to this schema.
3. **Implement schema filtering:** For large databases, sending the entire schema wastes context and confuses the model. Build a lightweight relevance filter: embed the user question, embed table/column descriptions, and include only tables above a similarity threshold. This keeps the prompt focused and improves generation accuracy.
4. **Handle ambiguity explicitly:** When the user's question maps to multiple possible interpretations (e.g., "revenue" could mean gross or net), return a structured clarification request instead of guessing. List the interpretations with example queries for each. This is a production feature most demos skip.

Phase 2: Build the SQL Generation and Safety Layer (Day 3–6)

1. **Generate SQL with structured output:** Use the instructor library or function calling to ensure the LLM returns: the SQL query, a natural language explanation of what it does, a confidence score, and a list of tables and columns accessed. Validate the SQL syntax before execution using sqlparse.
2. **Implement the guardrail middleware:** Before any query executes, pass it through a safety layer that: blocks all DDL (CREATE, ALTER, DROP), blocks all DML writes (INSERT, UPDATE, DELETE), enforces a row limit (LIMIT 1000 if none specified), rejects queries with subqueries deeper than 3 levels, and blocks queries estimated to scan more than N rows using EXPLAIN. Make each rule configurable and log every blocked query with the reason.
3. **Add query sandboxing:** Execute all generated queries in a read-only transaction that rolls back automatically. Use a database user with SELECT-only permissions as a second layer of defense. Even if the guardrail layer misses something, the database permissions prevent damage.
4. **Build the execution layer:** Run the validated SQL, capture results as a DataFrame, and package the response with: raw results (capped at the row limit), execution time, rows returned, and the EXPLAIN plan. Log everything for auditability.

Phase 3: Build the Hallucination Detection System (Day 6–9)

1. **Implement SQL-to-question verification:** After generating the SQL, send the query back to the LLM with the prompt: “What question does this SQL query answer?” Compare the back-translated question to the original. If they diverge significantly, the SQL probably doesn’t answer the right question. Score the alignment and flag low-confidence translations.
2. **Add result sanity checking:** After execution, perform basic sanity checks: are aggregated values within plausible ranges? Do counts match expected magnitudes? Are date ranges within the data’s timespan? Are there NULL-heavy columns that might indicate a bad JOIN? Flag anomalies with specific explanations.
3. **Build the multi-query validation:** For complex questions, generate two different SQL approaches independently (e.g., using different JOIN strategies or aggregation methods). Execute both. If results match, confidence is high. If they diverge, flag the discrepancy and present both results with explanations. Agreement between independent approaches is a strong correctness signal.
4. **Create a confidence scoring system:** Combine signals into a single confidence score: SQL syntax validity, back-translation alignment, result sanity check pass rate, multi-query agreement, and schema coverage (did the query use the tables/columns you’d expect for this question type?). Display confidence prominently alongside every result.

Phase 4: Build the Query Interface (Day 9–11)

1. **Build the API endpoints:** POST /v1/query accepts a natural language question and returns: the generated SQL, execution results, confidence score, and any guardrail warnings. GET /v1/schema returns the database schema. GET /v1/history returns past queries and results for the session.
2. **Create a Streamlit or React frontend:** A clean interface with: a text input for natural language questions, the generated SQL displayed with syntax highlighting (editable for power users), results in a sortable data table, the confidence score with a breakdown of contributing signals, and a history panel showing past queries.
3. **Add a feedback loop:** Let users mark results as correct or incorrect. Store this feedback alongside the query. Incorrect results become new test cases for the eval suite. Correct results become new few-shot examples that improve future generation. This is the flywheel.

Phase 5: Build the Evaluation Suite (Day 11–13)

1. **Create a golden query dataset:** Write 50+ natural language questions with verified correct SQL and expected results. Include: simple lookups, multi-table JOINs, aggregations with GROUP BY, date range filters, questions with ambiguous phrasing, and questions the database cannot answer. This is your regression suite.

2. **Run automated evals:** For each test case, measure: SQL exact match (does the generated SQL match the golden query?), execution match (do results match regardless of SQL approach?), hallucination detection rate (does the system correctly flag bad queries?), and guardrail effectiveness (are dangerous queries blocked?).
3. **Containerize and document:** Docker-compose with: PostgreSQL seeded with sample data, the FastAPI service, and the frontend. Include a README that leads with the eval numbers: "X% execution accuracy, Y% hallucination detection rate, zero unsafe queries executed across Z test cases."

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record the demo:** Show: a natural language question being translated to SQL, the guardrail blocking a dangerous query, the hallucination detector catching a bad translation, and the multi-query validation resolving a discrepancy. Under 4 minutes.
2. **Write the narrative:** Frame it as: "I built a text-to-SQL system with a X% accuracy rate that blocks 100% of destructive operations and detects Y% of hallucinated queries before they reach the user." Lead with safety. Companies care more about not breaking things than about accuracy percentages.

Project 9: Prompt Versioning and A/B Testing Platform

What You're Building: A platform that treats prompts as versioned artifacts (like code), lets teams deploy multiple prompt variants simultaneously, splits traffic between them, measures performance across custom metrics, and declares statistically significant winners — bringing the rigor of feature flagging and experimentation to LLM-powered features.

> **Why This Project Lands Interviews**
> Most AI teams change prompts by editing a string in production and hoping for the best. This project shows you understand that prompt engineering at scale is an experimentation problem, not a guessing game. It signals the kind of operational maturity that distinguishes senior AI engineers from junior ones.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Standard for ML tooling
LLM Provider	OpenAI / Anthropic	Multi-provider support
Storage	PostgreSQL	Versioning, experiments, results
Statistics	scipy.stats	Significance testing
API	FastAPI	Experiment management + serving

Dashboard	React or Streamlit	Experiment monitoring
Containerization	Docker + docker-compose	Full stack orchestration

Step-by-Step Build Guide

Phase 1: Build the Prompt Registry (Day 1–3)

1. **Design the versioning schema:** Every prompt gets a unique ID, a version number (auto-incrementing), the system prompt text, any few-shot examples, model parameters (temperature, max_tokens), a commit message explaining the change, and a timestamp. Store in PostgreSQL. This is git for prompts.
2. **Build the registry API:** POST /prompts creates a new prompt. POST /prompts/{id}/versions creates a new version. GET /prompts/{id}/versions lists all versions with diffs. GET /prompts/{id}/versions/{v} retrieves a specific version. Add a diff endpoint that shows exactly what changed between any two versions.
3. **Add rollback support:** Any version can be marked as the “active” version for production. Rolling back is just changing which version is active — no code deploy needed. Log every activation/deactivation event with who triggered it and why. This audit trail is critical for production trust.
4. **Implement prompt templates with variables:** Prompts should support template variables (e.g., {{user_name}}, {{context}}) that get filled at request time. The versioning system versions the template, not the filled prompt. Validate that all variables in the template are provided at serve time.

Phase 2: Build the Experiment Engine (Day 3–6)

1. **Design the experiment schema:** An experiment has: a name, the prompt ID being tested, two or more variant versions, traffic split percentages (e.g., 50/50 or 80/10/10), primary metric (what defines “better”), target sample size for significance, and status (draft/running/completed/cancelled).
2. **Build the traffic splitter:** When a request comes in for a prompt that has an active experiment, the splitter assigns the request to a variant based on the configured percentages. Use consistent hashing on a user ID or session ID so the same user always sees the same variant. Log the assignment.
3. **Implement the serving endpoint:** POST /v1/completions accepts a prompt ID and input variables. The system: resolves the active version (or experiment variant), fills the template, calls the LLM, logs the full request/response with the variant assignment, and returns the response. The caller doesn’t know or care about the experiment.
4. **Add guardrails for experiments:** Auto-stop experiments if a variant’s error rate exceeds a threshold (e.g., 10% API failures). Auto-stop if the losing variant is performing significantly worse than baseline on the primary metric. Notify the experiment owner when either condition triggers. This prevents bad prompts from affecting too many users.

Phase 3: Build the Metrics and Analysis Layer (Day 6–9)

1. **Define pluggable metric collectors:** Build a metric framework where teams can register custom evaluation functions. Built-in metrics: latency, token usage, cost, and error rate. Custom metrics: LLM-as-judge quality score, task-specific accuracy, user satisfaction signals. Each metric runs asynchronously after the response is returned.
2. **Implement statistical significance testing:** For each experiment, continuously calculate: the mean and variance of each metric per variant, a two-sample t-test (or Mann-Whitney U for non-normal distributions) comparing variants, the p-value and confidence interval, and the minimum detectable effect size given current sample size. Display whether the experiment has reached significance.
3. **Build the results dashboard:** For each running experiment, show: real-time metric comparison across variants (bar charts with confidence intervals), sample size progress toward the significance target, a time series of metric values to detect trends or instability, and a clear "winner/no winner/inconclusive" status with the supporting statistics.
4. **Add automated winner declaration:** When an experiment reaches statistical significance at the configured confidence level (default 95%), automatically: mark the winning variant, notify the experiment owner, and optionally auto-promote the winner to the active version. Include a 24-hour hold period before auto-promotion in case of data quality issues.

Phase 4: Build the Management Interface (Day 9–11)

1. **Create the experiment management UI:** A dashboard where teams can: create new experiments from the prompt registry, configure traffic splits and metrics, monitor running experiments, view completed experiment results with full statistical analysis, and promote winners to production with one click.
2. **Add prompt comparison tools:** A side-by-side view where you can pick any two prompt versions, send the same set of test inputs to both, and see outputs compared with automated quality scores. This is useful for quick sanity checks before launching a full experiment.
3. **Build the audit log:** Every action is logged: prompt creation, version changes, experiment start/stop, variant assignments, winner promotions, and rollbacks. This log answers "who changed the prompt that broke the feature last Tuesday?" — a question every AI team eventually needs to answer.

Phase 5: Integration and Testing (Day 11–13)

1. **Build a demo scenario:** Seed the system with a customer support email classifier. Create three prompt variants with meaningfully different approaches (e.g., zero-shot vs. few-shot vs. chain-of-thought). Run an experiment with 500+ synthetic requests. Let it converge to a winner.

2. **Containerize the full stack:** Docker-compose with: PostgreSQL, the FastAPI service, the experiment dashboard, and a worker for async metric computation. Include seed data so reviewers can see a completed experiment immediately.
3. **Write integration tests:** Test: prompt versioning and rollback, traffic splitting consistency (same user always gets same variant), metric collection accuracy, significance calculation correctness (use known distributions), and auto-stop on error rate spikes.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record the demo:** Walk through: creating a prompt, launching an experiment, watching metrics converge, the system declaring a winner with statistical backing, and promoting the winner to production. Show the audit log. Under 4 minutes.
2. **Write the narrative:** Frame it as: "I built a prompt experimentation platform that brings feature-flagging rigor to LLM development. Teams can test prompt changes against live traffic and get statistically significant results instead of guessing." Position it as infrastructure, not a toy.

Project 10: Fine-Tuning Pipeline with LoRA on a Domain-Specific Dataset

What You're Building: An end-to-end pipeline that takes a domain-specific dataset, applies LoRA (Low-Rank Adaptation) fine-tuning to an open-source base model, evaluates the fine-tuned model against the base model on task-specific benchmarks, and packages the result for deployment — with full experiment tracking and reproducibility.

> **Why This Project Lands Interviews**
> Fine-tuning is where AI engineering meets ML engineering. Most candidates either can't fine-tune at all or run a notebook once and call it done. Building a reproducible pipeline with evaluation and experiment tracking proves you can own the full model customization lifecycle that production AI teams need.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	ML ecosystem standard
Base Model	Llama 3 8B or Mistral 7B	Strong open-source options
Fine-Tuning	Hugging Face PEFT + TRL	Industry-standard LoRA tooling
Training	Unsloth or QLoRA	Memory-efficient on consumer GPU
Dataset	Custom domain-specific	Your unique value add
Eval	Custom + lm-eval-harness	Rigorous benchmarking

Experiment Tracking	Weights & Biases or MLflow	Reproducible runs
Deployment	vLLM or Ollama	Fast inference serving

Step-by-Step Build Guide

Phase 1: Build the Training Dataset (Day 1–4)

1. **Choose a specific domain and task:** Pick a narrow, well-defined task where you can demonstrate clear improvement over the base model. Examples: legal clause classification, medical note summarization, code review comment generation, or customer support tone matching. The domain doesn't need to be exotic — it needs to be measurable.
2. **Curate and format the training data:** Collect or create 500–2,000 high-quality examples in instruction-following format (instruction, input, output). Quality matters more than quantity — each example should represent the exact behavior you want the model to learn. Clean aggressively: remove duplicates, fix formatting inconsistencies, and verify correctness.
3. **Build train/validation/test splits:** Split your data 80/10/10. Ensure no data leakage between splits (especially important if examples were derived from the same source documents). The test set is sacred — never train on it, never tune hyperparameters against it. It's your honest performance measure.
4. **Create the evaluation benchmark:** Beyond the test split, create 30–50 handcrafted evaluation examples that cover edge cases, hard examples, and failure modes you expect. Score each one with task-specific metrics (not just perplexity). This benchmark is how you prove the fine-tuned model is better, not just different.

Phase 2: Set Up the Training Pipeline (Day 4–7)

1. **Configure LoRA parameters:** Set up PEFT with LoRA targeting the attention layers (q_proj, v_proj as baseline). Key hyperparameters: rank (start with 16), alpha (2x rank), dropout (0.05). Document why you chose these values. Use QLoRA (4-bit quantization) if running on a consumer GPU with limited VRAM.
2. **Build the training script with experiment tracking:** Write a configurable training script that logs to Weights & Biases (or MLflow): all hyperparameters, training loss curves, validation metrics at each checkpoint, GPU memory usage, and training duration. Every run should be reproducible from its config file alone.
3. **Implement early stopping and checkpointing:** Save model checkpoints at regular intervals. Track validation loss and stop training when it plateaus or starts increasing. This prevents overfitting and saves compute. Log which checkpoint was selected as the best and why.
4. **Run a hyperparameter sweep:** Test at least 3 configurations varying rank (8, 16, 32), learning rate (1e-4, 2e-4, 5e-4), and number of epochs (1, 3, 5). Log all runs to the

experiment tracker. Produce a comparison table showing how each configuration performs on the validation set. This data drives your final configuration choice.

Phase 3: Evaluate and Compare Models (Day 7–10)

1. **Run the base model against your benchmark:** Before any fine-tuning claims, establish the baseline. Run the base model (with the same prompt template) against your entire evaluation benchmark. Score every example. Store these results — they're the denominator in every improvement claim you make.
2. **Run the fine-tuned model against the same benchmark:** Using the best checkpoint from training, evaluate on the exact same benchmark with the exact same scoring criteria. Compare head-to-head on every example. Calculate: overall accuracy/quality improvement, per-category performance breakdown, examples where fine-tuning helped, and examples where it hurt (regressions).
3. **Add LLM-as-judge evaluation:** Beyond automated metrics, use a strong model (GPT-4o) as a judge to blindly compare base vs. fine-tuned outputs on each test example. Have it score quality on a 1–5 scale and explain its reasoning. This gives you a human-interpretable quality signal alongside automated metrics.
4. **Test for catastrophic forgetting:** Fine-tuning can degrade general capabilities. Run a subset of standard benchmarks (common sense QA, instruction following) on both the base and fine-tuned model. If general performance dropped significantly, you may need to reduce training epochs or adjust LoRA rank. Document the tradeoffs.

Phase 4: Build the Inference Pipeline (Day 10–12)

1. **Package the LoRA adapter for deployment:** Export the trained LoRA adapter weights (not the full model). The adapter is tiny (typically <100MB) while the base model is large (16GB+). This separation means you can swap adapters without reloading the base model — useful for serving multiple domain-specific models from one base.
2. **Set up inference serving:** Deploy using vLLM (for production-grade batched inference) or Ollama (for simpler local deployment). Load the base model, attach the LoRA adapter, and expose a chat completions API. Benchmark: tokens per second, latency P50/P95, and max concurrent requests.
3. **Build an A/B comparison endpoint:** Create an API that accepts a prompt and returns responses from both the base model and the fine-tuned model side by side. Include quality scores from the automated eval. This endpoint powers your demo and makes the improvement immediately visible.

Phase 5: Containerize and Document (Day 12–13)

1. **Package the full pipeline:** Create a reproducible setup with: a training script that can be re-run with a single command, a data processing pipeline that produces training-ready

datasets from raw inputs, an evaluation harness that benchmarks any model against your test suite, and an inference server with the fine-tuned model loaded.
2. **Write the experiment report:** A structured document showing: the problem and why fine-tuning was the right approach (vs. few-shot prompting or RAG), dataset statistics and curation methodology, hyperparameter sweep results, head-to-head comparison (base vs. fine-tuned) with specific examples, and catastrophic forgetting analysis.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record the demo:** Show: the base model struggling with a domain-specific task, the fine-tuned model handling it correctly, the W&B dashboard with training curves, and the A/B comparison endpoint with real examples. Under 4 minutes.
2. **Write the headline:** "I fine-tuned Llama 3 8B on [domain] data using LoRA and improved task performance from X% to Y% while maintaining Z% of general capabilities." Every number should be backed by the evaluation data in your repo.

Project 11: LLM Gateway with Rate Limiting, Fallback Routing, and Observability

What You're Building: A production API gateway that sits in front of all your organization's LLM calls, enforces per-team rate limits and budgets, automatically falls back to alternative providers when a primary provider has an outage or rate-limits you, and provides unified observability across every LLM interaction.

> **Why This Project Lands Interviews**
> Every company with more than one team using LLMs ends up building something like this. It's pure infrastructure engineering applied to AI — exactly the skill set that mid-level SWEs already have and that AI teams desperately need. This project lets you lead with your production engineering strengths while demonstrating deep AI systems knowledge.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+ or Go	High-throughput proxy layer
Proxy	FastAPI or net/http	Async-native request handling
Rate Limiting	Redis + token bucket	Distributed, sub-ms enforcement
Config	YAML + hot reload	No-deploy policy changes
Observability	OpenTelemetry + Prometheus	Industry-standard tracing
Dashboard	Grafana	Unified metrics visualization

Providers	OpenAI, Anthropic, Ollama	Multi-provider support
Containerization	Docker + docker-compose	Full stack orchestration

Step-by-Step Build Guide

Phase 1: Build the Unified Proxy Layer (Day 1–3)

1. **Build the provider abstraction:** Create a unified interface that normalizes requests across OpenAI, Anthropic, and Ollama. Incoming requests use a standard format. The gateway translates to each provider's API format, calls the provider, and translates the response back to the standard format. The caller never knows which provider served the request.
2. **Implement request authentication and routing:** Every request includes a team API key. The gateway validates the key, looks up the team's configuration (allowed models, rate limits, budget), and routes to the appropriate provider. Teams can be restricted to specific models or providers based on their plan.
3. **Add streaming passthrough:** Support both streaming and non-streaming responses. For streaming, the gateway acts as a transparent proxy: it forwards chunks from the provider to the client in real time while simultaneously logging the complete response for observability. This is critical for latency-sensitive applications.
4. **Handle request enrichment:** The gateway can inject standard system prompts, append compliance disclaimers, or add content filters before forwarding to the provider. Make this configurable per team. This centralizes policy enforcement without requiring every team to implement it themselves.

Phase 2: Build Rate Limiting and Budget Enforcement (Day 3–6)

1. **Implement token bucket rate limiting:** Use Redis to maintain per-team token buckets with configurable rates (requests per minute, tokens per minute). Enforce limits before forwarding the request. Return standard 429 responses with Retry-After headers when limits are hit. This must be atomic and distributed-safe.
2. **Add budget caps:** Each team gets a monthly or daily budget in dollars. Track spending by computing cost per request (input tokens * input price + output tokens * output price). When a team approaches their limit (80%), send a warning. When they hit it, block requests and return a clear error explaining the budget cap.
3. **Build tiered rate limits:** Different request types get different limits. A batch data processing job should have a lower priority than a real-time user-facing request. Implement priority queues so high-priority requests get served even when overall limits are near capacity.
4. **Create the admin API:** Endpoints for: viewing current rate limit status per team, adjusting limits and budgets without restart, viewing spending dashboards, and setting

up alerts when teams approach limits. All changes are logged with who made them and when.

Phase 3: Build the Fallback and Resilience Layer (Day 6–9)

1. **Implement health checking:** The gateway continuously monitors each provider's health: send lightweight test requests every 30 seconds, track error rates and latency P99 over rolling windows, and maintain a status (healthy/degraded/down) for each provider-model combination. Store health history for post-incident analysis.
2. **Build automatic fallback routing:** When a primary provider is degraded or down, automatically route requests to a configured fallback. Example: if OpenAI GPT-4o is timing out, route to Anthropic Claude Sonnet. Define fallback chains per model tier (not per specific model) so the system always finds an available option.
3. **Add retry with exponential backoff:** Before falling back to another provider, retry the primary with exponential backoff (up to 3 retries). Distinguish between retryable errors (rate limits, timeouts) and non-retryable errors (auth failures, content policy violations). Only fall back for retryable errors that exhaust retries.
4. **Implement circuit breakers:** If a provider fails more than N times in M seconds, open the circuit — stop sending requests to that provider entirely and route everything to fallbacks. After a cooldown period, send a single test request (half-open state). If it succeeds, close the circuit and resume normal routing. Log every state change.

Phase 4: Build the Observability Layer (Day 9–11)

1. **Instrument with OpenTelemetry:** Add distributed tracing spans for: request receipt, authentication, rate limit check, provider selection, LLM API call, response processing, and response delivery. Every span includes: team ID, model requested, model served, token counts, latency, and cost.
2. **Export Prometheus metrics:** Key metrics: requests per second (by team, model, provider), error rate (by team, model, provider, error type), latency percentiles (P50, P95, P99 by provider), token throughput (input + output tokens per second), cost per team per day, fallback trigger rate, and circuit breaker state changes.
3. **Build the Grafana dashboards:** Create three dashboards. Operations: provider health, error rates, fallback events, circuit breaker status. Business: per-team spending, budget utilization, usage trends. Performance: latency percentiles, token throughput, cache hit rates (if integrated with Project 7).
4. **Add alerting rules:** Configure alerts for: provider error rate above threshold, team approaching budget cap, latency P99 above SLA, and circuit breaker opening. Route alerts to Slack with actionable context: what happened, which teams are affected, and the auto-fallback status.

Phase 5: Integration Test and Load Test (Day 11–13)

1. **Build an integration test suite:** Test: rate limiting enforces correctly under concurrent load, budget caps block requests at the right threshold, fallback routing activates when primary fails (simulate failures), circuit breakers open and close correctly, and streaming responses pass through without corruption.
2. **Run a load test:** Send 5,000+ concurrent requests through the gateway with mixed team keys, models, and priorities. Measure: gateway overhead latency (should be <10ms), rate limiting accuracy under load, fallback behavior under simulated provider outages, and dashboard accuracy (do displayed metrics match reality?).
3. **Containerize the full stack:** Docker-compose with: the gateway service, Redis, Prometheus, Grafana (with pre-configured dashboards), and mock provider endpoints for testing. Include a setup script that creates demo teams with different rate limits so reviewers can see the system in action immediately.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record the demo:** Show: requests flowing through the gateway with real-time Grafana metrics, a simulated provider outage triggering fallback routing, rate limiting kicking in for a high-volume team, and the circuit breaker opening and recovering. Under 4 minutes.
2. **Write the narrative:** Frame it as: “I built an LLM API gateway handling X requests/second with <10ms overhead, automatic multi-provider failover, and per-team budget enforcement.” This is infrastructure. Lead with the reliability numbers and the operational problem it solves.

Project 12: AI Feature Flag System with Gradual Rollout and Quality Monitoring

What You’re Building: A feature flag platform specifically designed for AI-powered features that supports gradual percentage-based rollouts, automatically monitors quality metrics during rollout, and triggers automatic rollback if the AI feature’s output quality degrades below a configurable threshold.

> **Why This Project Lands Interviews**
> Every engineering team uses feature flags for traditional software. Almost none have adapted the pattern for AI features, where “working” isn’t binary — it’s a quality gradient. This project proves you understand that shipping AI features requires different operational patterns than shipping traditional code, and you can build the tooling to support that.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Standard for AI tooling

Storage	PostgreSQL + Redis	Durable config + fast evaluation
Quality Eval	Custom + LLM-as-judge	Real-time quality scoring
SDK	Python client library	Drop-in integration for apps
Dashboard	React or Streamlit	Rollout monitoring
Alerting	Slack Webhooks	Rollback notifications
Containerization	Docker + docker-compose	Full stack orchestration

Step-by-Step Build Guide

Phase 1: Build the Flag Evaluation Engine (Day 1–3)

1. **Design the AI feature flag schema:** Beyond traditional boolean flags, AI flags need: a rollout percentage (0–100%), a quality threshold (minimum acceptable quality score), a rollback trigger (conditions that cause automatic rollback), a baseline configuration (what to serve when the flag is off — e.g., the previous prompt version or a non-AI fallback), and the experimental configuration (the new AI feature being tested).
2. **Build the evaluation SDK:** A lightweight Python client that applications import. The core method: flag_client.evaluate(flag_name, user_context) returns which variant to serve. The SDK handles: consistent user assignment (same user always gets same variant via hashing), percentage-based rollout, local caching of flag configurations, and graceful degradation if the flag service is unreachable (default to baseline).
3. **Implement targeting rules:** Beyond random percentage splits, support targeting: by user segment (e.g., internal users first, then beta users, then everyone), by geography, by request metadata (e.g., only enable for certain input types), and allowlist/blocklist for specific user IDs. This lets teams do careful, controlled rollouts.
4. **Build the flag management API:** CRUD endpoints for flags, plus: POST /flags/{id}/rollout to update the percentage, POST /flags/{id}/pause to halt the rollout at the current percentage, and POST /flags/{id}/rollback to immediately switch all traffic to baseline. All changes are logged with the actor and reason.

Phase 2: Build the Quality Monitoring Layer (Day 3–6)

1. **Define quality metrics per flag:** Each AI feature flag specifies how to measure quality. Built-in options: LLM-as-judge scoring (send each output to a judge model for a 1–5 rating), user feedback signals (thumbs up/down, explicit ratings), latency thresholds (AI features that are too slow degrade UX even if output is good), and error rate (API failures, malformed outputs, timeout rates).
2. **Build the async quality evaluator:** After every AI-flag-gated response, queue an async quality evaluation. The evaluator scores the output using the flag's configured metrics and writes the score to the database. This must not add latency to the user-facing response. Use a background worker with a message queue.

3. **Implement rolling quality windows:** Track quality scores in rolling windows (last 100 evaluations, last 1 hour, last 24 hours). Calculate: mean quality score, standard deviation, P10 quality (worst 10th percentile), and the quality trend (improving, stable, degrading). Compare experimental vs. baseline scores continuously.
4. **Build the automatic rollback trigger:** When the experimental variant's quality drops below the flag's configured threshold for a sustained period (e.g., P10 below 3.0 for more than 50 consecutive evaluations), automatically: set the rollout percentage to 0%, send a Slack alert with the quality data, and log the rollback event with full context. Include a cooldown to prevent flapping.

Phase 3: Build Gradual Rollout Automation (Day 6–9)

1. **Implement staged rollout schedules:** Define a rollout plan as a series of stages: 1% for 2 hours, 5% for 6 hours, 25% for 24 hours, 50% for 24 hours, 100%. At each stage boundary, the system checks quality metrics. If quality is above threshold, it auto-advances to the next stage. If quality dips, it pauses and alerts.
2. **Add canary analysis:** At each rollout stage, compare the experimental variant's metrics against the baseline. Use statistical testing (similar to Project 9) to determine if there's a significant quality difference. The rollout only advances when the experimental variant is statistically no worse than baseline.
3. **Build the rollout dashboard:** For each active rollout, show: current stage and percentage, quality metric comparison (experimental vs. baseline) over time, upcoming stage transitions and their conditions, any triggered pauses or rollbacks with reasons, and estimated time to full rollout based on current quality trends.
4. **Support dark mode testing:** Before any real rollout, support "shadow mode": run the experimental variant on all traffic but don't show results to users. Instead, log what would have been shown and evaluate quality offline. This catches catastrophic failures before any user is affected.

Phase 4: Build the Dashboard and Integration (Day 9–11)

1. **Create the flag management UI:** A dashboard showing: all flags and their current status (off/rolling out/fully on/rolled back), real-time quality metrics for active flags, the rollout schedule with progress indicators, and a one-click rollback button with confirmation.
2. **Build the analytics view:** Historical data for each flag: quality metrics over the full rollout history, the impact of the AI feature on business metrics (if tracked), a summary of all rollback events and their causes, and the time-to-full-rollout for successful features.
3. **Write the client SDK documentation:** Show exactly how a developer integrates the flag into their application with code examples. Emphasize that the integration is minimal (3–5 lines of code) and the quality monitoring happens automatically. Include examples for common patterns: AI vs. non-AI fallback, prompt version A/B testing, and model swap testing.

Phase 5: Integration Testing and Demo (Day 11–13)

1. **Build a demo application:** Create a simple app (e.g., an AI-powered email subject line generator) that uses the flag system. Show: the flag starting at 0%, gradual rollout with quality monitoring, the system detecting a deliberately bad prompt variant and auto-rolling back, and a successful rollout of a good variant to 100%.
2. **Write integration tests:** Test: consistent user assignment across evaluations, automatic rollback triggers correctly on quality degradation, staged rollout advances correctly on quality thresholds, and the SDK gracefully handles flag service outages.
3. **Containerize everything:** Docker-compose with: PostgreSQL, Redis, the flag service API, the background quality evaluator, the dashboard, and the demo application. Include a script that runs the full rollout demo scenario end to end.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record the demo:** Show the full lifecycle: flag created, rollout started at 1%, quality checked and advancing through stages, a bad variant detected and auto-rolled back with Slack alert, a good variant reaching 100%. Under 4 minutes.
2. **Write the narrative:** Frame it as: “I built a feature flag system for AI that catches quality regressions during rollout and auto-rolls back before users are impacted. Traditional feature flags can’t do this because AI features fail on a gradient, not a binary.” That distinction is the whole point.

Project 13: Automated Eval Dataset Generator from Production Logs

What You’re Building: A system that continuously mines production LLM logs, identifies interesting, edge-case, and failure-mode interactions, and automatically converts them into labeled evaluation test cases — building an ever-growing, production-representative eval dataset without manual curation.

> **Why This Project Lands Interviews**
> The hardest part of AI evaluation isn’t building the eval harness — it’s building the dataset. Most teams rely on hand-curated golden sets that go stale. You’re solving the data supply problem by turning production traffic into eval data automatically. This is a force multiplier that AI teams dream about.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Standard for ML pipelines

Log Storage	PostgreSQL or ClickHouse	Queryable log warehouse
Clustering	scikit-learn + HDBSCAN	Interaction pattern discovery
LLM	GPT-4o or Claude Sonnet	Labeling and quality assessment
Eval Runner	Custom harness	Run evals against the dataset
Dashboard	Streamlit	Dataset explorer and curator
Scheduler	Cron or Celery	Automated nightly processing
Containerization	Docker + docker-compose	Full pipeline orchestration

Step-by-Step Build Guide

Phase 1: Build the Log Ingestion and Normalization Layer (Day 1–3)

1. **Design the log schema:** Each production log entry captures: the user prompt, system prompt, model used, raw LLM response, latency, token counts, any user feedback signals (thumbs up/down, edits, retries), the application feature that generated this call, and a timestamp. Normalize across all LLM call sites into this unified format.
2. **Build the ingestion pipeline:** Create adapters for common logging sources: structured JSON logs, OpenTelemetry traces, and direct database reads. The pipeline deduplicates, validates schema compliance, and writes to the central log store. Handle PII by offering configurable redaction rules (regex patterns, named entity detection) that clean sensitive data before storage.
3. **Implement sampling strategies:** You don't need every log entry — you need a representative sample. Build three sampling modes: uniform random (baseline diversity), stratified by feature/model/error-status (ensures all segments are represented), and signal-boosted (oversample entries with negative user feedback, retries, or high latency). Make sampling configurable per pipeline run.

Phase 2: Build the Interaction Classifier (Day 3–6)

1. **Cluster interactions by semantic similarity:** Embed all sampled prompts and cluster them using HDBSCAN. This reveals the natural categories of questions your system handles. Name each cluster based on representative examples. Track cluster sizes to understand your traffic distribution.
2. **Identify edge cases and anomalies:** Flag interactions that are outliers: prompts that don't cluster well (novel requests), responses with low confidence scores, interactions where the user immediately retried with a rephrased question, unusually long or short responses, and responses that triggered content filters. These outliers are your most valuable eval candidates.
3. **Categorize interaction quality:** Use an LLM-as-judge to assess each sampled interaction: was the response helpful, accurate, and complete? Assign a quality score (1–5). High-quality interactions become positive test cases (verify the model keeps doing

this well). Low-quality interactions become negative test cases (verify future models fix this).

4. **Build a difficulty estimator:** Classify each interaction by difficulty: simple (direct question, clear answer), moderate (requires reasoning or multi-step response), hard (ambiguous, multi-part, or requires domain expertise), and adversarial (deliberately tricky, prompt injection attempts, or edge-case inputs). Difficulty labels help balance the eval dataset.

Phase 3: Build the Auto-Labeling Pipeline (Day 6–9)

1. **Generate golden answers for test cases:** For each candidate test case, generate a reference answer using a strong model (GPT-4o) with chain-of-thought reasoning. For factual questions, verify against source data if available. For subjective questions, generate a quality rubric instead of a single golden answer. Store both the reference and the rubric.
2. **Create multi-dimensional labels:** Each test case gets labeled on: expected quality score (1–5), expected behavior (should answer, should refuse, should ask for clarification), key assertions the response must contain, key assertions the response must NOT contain (hallucination traps), and the difficulty and category from the classifier.
3. **Implement confidence-based routing:** Not all auto-generated labels are trustworthy. When the labeling model's confidence is high (strong agreement between multiple labeling runs), add the test case to the dataset automatically. When confidence is low, route to a human review queue. This keeps the dataset growing without sacrificing quality.
4. **Build deduplication against existing eval data:** Before adding a new test case, check similarity against all existing test cases. If a near-duplicate exists (cosine similarity > 0.92), skip it. Track coverage metrics: which clusters are well-represented and which need more examples? Prioritize generation for under-represented categories.

Phase 4: Build the Eval Runner and Regression Tracker (Day 9–11)

1. **Build the eval harness:** A configurable runner that: takes any model endpoint, runs the full eval dataset against it, scores each response using the stored rubrics and metrics, and produces a structured report with pass/fail rates per category, per difficulty level, and per metric dimension.
2. **Implement regression detection:** Compare each eval run against the previous run. Flag: new failures (test cases that used to pass but now fail), new passes (cases that were failing but are now fixed), score changes above a threshold, and category-level performance shifts. This is the early warning system for model degradation.
3. **Track dataset growth over time:** Dashboard showing: total test cases over time, distribution by category and difficulty, auto-labeled vs. human-reviewed counts, coverage heatmap (which clusters are well-tested vs. sparse), and the eval dataset's "freshness" (what percentage of test cases came from the last 30 days of production traffic).

Phase 5: Build the Curation Dashboard (Day 11–13)

1. **Create the dataset explorer:** A UI where reviewers can: browse test cases by category, difficulty, and quality, view the full interaction (prompt, response, labels), approve, edit, or reject auto-generated labels, and add manual annotations or notes. This is where the human-in-the-loop oversight happens.
2. **Build the review queue:** Low-confidence auto-labels land in a review queue. Show reviewers: the candidate test case, the auto-generated labels with confidence scores, similar existing test cases for context, and quick-action buttons (approve as-is, edit labels, reject). Track reviewer throughput and inter-annotator agreement.
3. **Containerize and schedule:** Docker-compose with: the log store, the classification pipeline, the auto-labeling service, the eval runner, and the curation dashboard. Set up a nightly cron job that runs the full pipeline: sample new logs, classify, auto-label, deduplicate, and add to the dataset. The eval suite grows while you sleep.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record the demo:** Show: production logs being processed, the clustering revealing interaction patterns, auto-labeling generating test cases, the eval runner catching a regression, and the dataset growth over time. Under 4 minutes.
2. **Write the narrative:** Frame it as: “I built a system that automatically converts production LLM logs into a growing eval dataset. In two weeks of simulated production traffic, it generated X test cases across Y categories with Z% auto-label accuracy.” The self-growing aspect is the headline.

Project 14: Multi-Modal Document Processor with OCR, LLM Extraction, and Validation

What You’re Building: An end-to-end document processing pipeline that accepts any document format (PDF, image, scan), performs OCR to extract raw text, uses LLMs to extract structured data from the text, and validates every extraction against configurable business rules — with a human-in-the-loop review interface for low-confidence results.

> **Why This Project Lands Interviews**
> Document processing is one of the largest categories of enterprise AI deployment. It combines computer vision (OCR), natural language understanding (extraction), and production engineering (validation, error handling, scale). This project touches every layer of the AI engineering stack and solves a problem every large company is actively paying to solve.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	ML ecosystem standard
OCR	Tesseract + EasyOCR	Open-source, multi-engine
Vision Model	GPT-4o vision or Claude	Direct image understanding
LLM Extraction	GPT-4o + instructor	Structured output enforcement
Validation	Pydantic + custom rules	Type-safe business rules
Queue	Celery + Redis	Async document processing
Review UI	React or Streamlit	Human-in-the-loop interface
Containerization	Docker + docker-compose	Full pipeline orchestration

Step-by-Step Build Guide

Phase 1: Build the Ingestion and OCR Layer (Day 1–3)

1. **Build a multi-format document loader:** Accept PDFs (native text and scanned), images (JPEG, PNG, TIFF), and scanned documents. For PDFs, first try native text extraction (PyMuPDF). If the PDF has images or the text extraction yields garbage, fall back to OCR. Detect the right strategy automatically by checking text density per page.
2. **Implement dual-engine OCR:** Run both Tesseract and EasyOCR on every scanned page. Compare outputs. When they agree, confidence is high. When they disagree, use character-level alignment to identify discrepancies and pick the higher-confidence reading for each segment. This ensemble approach significantly reduces OCR errors.
3. **Add vision model fallback:** For documents where OCR struggles (handwriting, poor scan quality, complex layouts like tables), send the page image directly to GPT-4o vision or Claude vision. Ask the model to extract text preserving the document's structure. This is more expensive but handles edge cases that traditional OCR cannot.
4. **Build the preprocessing pipeline:** Before OCR, apply: deskewing (straighten rotated scans), binarization (convert to high-contrast black and white), noise removal, and resolution upscaling for low-quality scans. Track which preprocessing steps were applied and their effect on OCR confidence for each page.

Phase 2: Build the LLM Extraction Engine (Day 3–6)

1. **Define extraction schemas per document type:** Create Pydantic models for each document type you process. For invoices: vendor name, invoice number, line items (description, quantity, unit price, total), tax, total amount, payment terms, due date. For contracts: parties, effective date, term, key obligations, termination clauses. Make schemas configurable and extensible.
2. **Build the extraction pipeline:** For each document, first classify its type (using the OCR text + an LLM classifier). Then load the corresponding extraction schema. Send the

OCR text to the LLM with: the schema definition, 2–3 few-shot examples for that document type, and explicit instructions to output only information present in the text (no inference, no defaults). Use the instructor library to enforce the Pydantic schema in the LLM output.

3. **Implement chunk-and-merge for long documents:** Documents longer than the context window need to be processed in chunks. Split by page or section, extract from each chunk independently, then merge results. Handle conflicts: if two chunks extract different values for the same field, flag the conflict and include both values with their source locations.
4. **Add extraction confidence scores:** For each extracted field, compute a confidence score based on: how clearly the value appeared in the OCR text (exact match vs. fuzzy), the LLM's self-reported confidence, whether multiple chunks agreed on the value, and whether the value passes format validation (e.g., dates parse correctly, amounts are numeric). Return per-field confidence alongside the extraction.

Phase 3: Build the Validation and Business Rules Engine (Day 6–9)

1. **Implement type-level validation:** Using Pydantic validators, enforce: dates are valid and within plausible ranges, monetary amounts are positive and correctly formatted, required fields are present, enums match allowed values, and cross-field consistency (e.g., line item totals sum to the invoice total). Return specific validation error messages per field.
2. **Add business rule validation:** Beyond type checking, implement domain-specific rules. For invoices: does the vendor exist in the known vendor list? Is the total within expected ranges for this vendor? Are payment terms standard? For contracts: is the effective date in the future? Are all required clause types present? Are any termination clauses unusually broad?
3. **Build the anomaly detector:** Compare each extraction against historical data for that document type and source. Flag statistical outliers: amounts significantly higher or lower than typical, unusual vendor names, dates that don't follow the expected pattern, and any fields that changed dramatically from previous documents from the same source.
4. **Create the confidence-based routing:** Based on the overall confidence score and validation results, route each document: high confidence + all validations pass → auto-approve, medium confidence or minor validation warnings → human review queue (fast review), low confidence or critical validation failures → human review queue (detailed review). Track auto-approval rates and accuracy over time.

Phase 4: Build the Human Review Interface (Day 9–11)

1. **Create the review dashboard:** A side-by-side interface showing: the original document (rendered as an image) on one side and the extracted data on the other. Highlight the source location in the document for each extracted field. Color-code fields by confidence: green (high), yellow (medium), red (low/failed validation).

2. **Build inline editing:** Reviewers can click any extracted field to edit it. When edited, log: the original extracted value, the corrected value, who corrected it, and whether the original was wrong (extraction error) or the business rule was too strict (validation false positive). This data feeds back into improving both extraction and validation.
3. **Implement batch review workflows:** For high-volume processing, build a queue-based workflow: reviewers see a prioritized list of documents needing review, can approve/reject/edit in bulk, and see their throughput and accuracy stats. Include keyboard shortcuts for common actions to maximize reviewer efficiency.

Phase 5: Build Feedback Loops and Analytics (Day 11–13)

1. **Feed corrections back to the extraction pipeline:** Every human correction becomes a training signal. Accumulate corrections and periodically: update few-shot examples with corrected examples, identify systematic extraction errors and adjust prompts, tune confidence thresholds based on actual accuracy, and generate reports on where the pipeline fails most.
2. **Build the processing analytics dashboard:** Show: documents processed per day/week, auto-approval rate over time (the key efficiency metric), extraction accuracy by document type and field, average review time per document, and OCR engine performance comparison. These metrics tell the story of the pipeline's operational maturity.
3. **Containerize the full pipeline:** Docker-compose with: the ingestion API, OCR workers, LLM extraction workers, the validation service, the review UI, PostgreSQL, Redis, and Celery workers. Include sample documents for demo purposes.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record the demo:** Show: uploading a scanned invoice, OCR extracting text, LLM extracting structured data, validation catching an anomaly, the reviewer correcting a field, and the analytics dashboard. Under 4 minutes.
2. **Write the narrative:** Frame it as: "I built a document processing pipeline that auto-extracts structured data from scanned documents with X% accuracy, auto-approves Y% of documents without human review, and reduces manual processing time by Z%." Lead with the efficiency numbers.

Project 15: Agent Orchestration System with Tool Use, Memory, and Human-in-the-Loop

What You're Building: A multi-agent orchestration platform where a supervisor agent decomposes complex tasks, delegates subtasks to specialized tool-using agents, maintains persistent memory across interactions, and escalates to a human operator when confidence is low or the task requires approval — with full observability into every agent decision.

Why This Project Lands Interviews

Agents are the frontier of AI engineering, and most candidate projects are single-agent toy demos. Building a multi-agent system with real tool use, persistent memory, and human-in-the-loop escalation proves you can architect the kind of autonomous AI systems that companies are actively building and hiring for right now.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Ecosystem standard
Orchestration	LangGraph	State machine for agent workflows
LLM Providers	OpenAI + Anthropic	Multi-model agent routing
Tool Framework	Custom + MCP	Extensible tool integration
Memory	PostgreSQL + ChromaDB	Short-term + semantic long-term
Queue	Redis + Celery	Async task execution
Review UI	React or Streamlit	Human-in-the-loop interface
Containerization	Docker + docker-compose	Full system orchestration

Step-by-Step Build Guide

Phase 1: Build the Agent Architecture (Day 1–4)

1. **Design the agent hierarchy:** Create three layers. The Supervisor Agent receives complex tasks, creates execution plans, and delegates to specialists. Specialist Agents each own a domain (research, data analysis, writing, code execution) and have access to domain-specific tools. The Reviewer Agent validates outputs from specialists before returning to the supervisor. Model each agent as a LangGraph node with defined input/output schemas.
2. **Build the task decomposition engine:** The supervisor's core capability: take a complex request and break it into an ordered list of subtasks, each assigned to a specialist. Include dependencies (subtask B needs the output of subtask A). Use structured output to enforce a valid execution plan with: subtask description, assigned specialist, required inputs, expected output format, and estimated complexity.
3. **Implement the tool registry:** Build a registry where tools are registered with: a name, description, input/output schemas, the specialist agents that can use them, and rate limits. Start with: web search, file read/write, code execution (sandboxed), database query, and API calls. Each tool invocation is logged with inputs, outputs, latency, and success/failure.
4. **Build the LangGraph state machine:** Wire the agents into a LangGraph graph with: task intake → planning → parallel/sequential specialist execution → review → synthesis

→ delivery. Include conditional edges: if a specialist fails, retry with a different approach; if the reviewer rejects output, route back to the specialist with feedback; if confidence is low, route to human escalation.

Phase 2: Build the Memory System (Day 4–7)

1. **Implement short-term working memory:** During a task execution, all agents share a working memory store: the current execution plan, outputs from completed subtasks, intermediate results, and error logs. Store in Redis for fast access. This memory is scoped to a single task and cleared when the task completes.
2. **Build long-term semantic memory:** After task completion, extract and embed key information: what the user asked for, what approach worked, what tools were used, any domain-specific facts discovered, and user preferences observed. Store in ChromaDB. Future tasks can query this memory to inform planning. This is how the system gets smarter over time.
3. **Implement memory retrieval for planning:** When the supervisor creates an execution plan, it first queries long-term memory for: similar past tasks and their execution plans, approaches that worked well (and those that didn't), user-specific preferences and context, and relevant domain facts. Retrieved memories are injected into the planning prompt.
4. **Add memory management:** Implement: memory importance scoring (frequently accessed memories are more important), memory consolidation (merge similar memories into higher-level summaries), memory expiration (stale memories decay over time), and a memory dashboard showing what the system "remembers" about each user. Include a delete endpoint for user data requests.

Phase 3: Build the Human-in-the-Loop System (Day 7–10)

1. **Define escalation triggers:** The system escalates to a human when: the supervisor's confidence in its plan is below a threshold, a specialist fails twice on the same subtask, the task involves sensitive operations (financial transactions, data deletion, external communications), the reviewer's quality score for a deliverable is below threshold, or the user explicitly requests human review.
2. **Build the approval queue:** When escalation triggers, the system: pauses execution at the current step, packages the full context (original task, plan, completed steps, current step that needs approval, the agent's proposed action), pushes it to a review queue, and notifies the human reviewer. The system waits for approval, rejection, or modification before continuing.
3. **Implement granular approval levels:** Not all escalations need the same review depth. Define levels: Notify (proceed but inform the human), Approve action (human confirms the next step), Approve plan (human reviews the full execution plan before any work begins), and Take over (human directly provides the output, agents stand down). Map escalation triggers to appropriate levels.

4. **Build the review interface:** A UI showing: the task context and execution progress, the specific decision point requiring human input, the agent's proposed action with its reasoning, relevant memories and past similar decisions, and action buttons (approve, modify, reject, take over). Include a chat panel for the human to ask the agent clarifying questions before deciding.

Phase 4: Build Observability and Debugging (Day 10–12)

1. **Implement full execution tracing:** Every task execution produces a trace tree: the supervisor's planning decisions, each specialist's tool calls and reasoning steps, the reviewer's evaluations, memory retrievals and their influence on decisions, and human escalation events and resolutions. Use OpenTelemetry spans with custom attributes.
2. **Build the trace explorer UI:** A visual representation of the agent workflow as a tree/graph. Each node shows: the agent that acted, what it decided, what tools it called, latency and cost, and any errors. Color-code by status (success/warning/failure/escalated). Clicking a node expands the full context including the LLM prompt and response.
3. **Add cost and performance tracking:** Per task, track: total LLM tokens used (by agent and model), total tool calls, total wall-clock time, human review time (if escalated), and total cost. Aggregate across tasks to show: cost per task type, most expensive agents, tool usage patterns, and escalation rate trends.
4. **Build the replay system:** For debugging, allow replaying any past task execution: load the original task and context, step through each agent decision, modify any input at any step and see how the execution diverges, and compare the replayed execution against the original. This is invaluable for diagnosing failures and testing improvements.

Phase 5: Integration and End-to-End Testing (Day 12–13)

1. **Build a compelling demo scenario:** Design a complex task that showcases the full system: a research task that requires web search, data extraction, analysis, and a written summary. Show: the supervisor decomposing the task, specialists working in parallel, the reviewer catching an issue and sending it back, memory informing a decision, and a human approving the final deliverable.
2. **Containerize the full system:** Docker-compose with: the orchestration API, Redis (working memory), PostgreSQL (persistent state), ChromaDB (long-term memory), Celery workers (async specialists), the trace explorer UI, and the human review UI. Include a demo script that runs the showcase scenario automatically.
3. **Write end-to-end tests:** Test: task decomposition produces valid plans, specialists correctly use their tools, the reviewer catches deliberately bad outputs, memory retrieval improves planning for repeated similar tasks, human escalation triggers at the right moments, and the system recovers gracefully from agent failures.

Phase 6: Polish for Portfolio (Day 13–14)

1. **Record the demo:** Show the full task lifecycle: complex request in, supervisor planning, specialists executing with tool calls, reviewer validating, human approving a sensitive step, memory saving lessons learned, and the trace explorer showing every decision. Under 5 minutes.
2. **Write the narrative:** Frame it as: “I built a multi-agent orchestration system where AI agents decompose complex tasks, use tools to execute them, learn from past interactions via persistent memory, and escalate to humans when confidence is low. It’s not an AI demo — it’s production infrastructure for autonomous AI workflows.” Lead with the architecture diagram showing the agent hierarchy and decision flow.